Shrink First, Adapt Cautiously: A Training-Free Hierarchical Predictor with Leave-One-Out-Gated Test-Time Adaptation for the Predictive AI Evaluation Competition

[Author Name]
[Affiliation]
neelsirivara@outlook.com

Abstract

The Predictive AI Evaluation Competition (PAIEC) asks for the probability that an AI system (the *subject*) answers a benchmark *item* correctly, given the subject's visible attributes, the item text, and between 0 and 31 labels acquired on the same subject–benchmark pair. Scoring is the Brier area-under-the-learning-curve (ALC), which places 60% of its weight on budgets of at most seven labels. We describe a submission built, for most of the project, without the official training release (`measurement-db` is gated; credentials became available two days before the deadline), i.e. a predictor with no fitted parameters: a hand-set attribute prior at budget zero, beta-binomial shrinkage of the pair success rate towards that prior, and two item-level adaptation components—a ridge-logistic adjustment on cheap item features and a TF-IDF nearest-neighbour correction—that are switched on only when a leave-one-out test on the acquired labels shows a clear improvement. A streaming acquisition policy removes near-duplicate candidates. Because the test data are private, we reproduce the official protocol locally on a real testbed of 17 pairs (four small open models on GSM8K, SVAMP and BIG-Bench-Hard) and on a synthetic IRT testbed with oracle bounds: the shipped predictor reaches a Brier ALC of 0.2224 (95% CI 0.1949–0.2454) against 0.2871 for the organisers' empirical-mean baseline and 0.25 for the constant, and 0.1797 against 0.2208 on the synthetic testbed. Oracle analysis splits the remaining headroom into rate estimation at small budgets, which only a better prior can reduce, and item-level discrimination, which 31 labels cannot deliver: the gated item-level components change the ALC by +0.0000 (synthetic) and -0.0004 (real), sentence embeddings did not beat TF-IDF, and uncertainty sampling did not beat random acquisition. When the release became available we used it three times: to validate the shipped rule on 16,303 real subject–benchmark pairs from 144 benchmarks with the competition protocol in closed form (ALC 0.2051 against 0.2396 for the empirical mean and 0.2470 for the constant), to fit the attribute prior, and, under a pre-registered ship rule, to train an item-text difficulty prior on other benchmarks' items. Under benchmark-held-out cross-validation neither beat the hand-set prior (text prior: ALC change -0.0000 [-0.0013, +0.0014]; its out-of-fold slope was negative in every fold), so nothing fitted is shipped. The predictor is pure standard-library Python, deterministic, and runs at about 1,010 calls per second on one CPU core; the submission ZIP passes the organisers' validator. Code and text were produced with the help of an LLM coding assistant (Claude).

Technical report, Predictive AI Evaluation Competition (PAIEC), 40th Conference on Neural Information Processing Systems (NeurIPS 2026).

1 Introduction

Evaluating a new AI system on the full set of available benchmarks is expensive, and the number of benchmarks keeps growing [Liang et al., 2023]. A line of recent work therefore asks how well performance can be *predicted* rather than measured: from a few hundred carefully chosen items [Polo et al., 2024, Perlitz et al., 2024], from the scores of other models [Ruan et al., 2024], or from a handful of labelled instances of the new model itself [Pacchiardi et al., 2024]. PAIEC [Stanford AIMS Lab, 2026] turns this into a forecasting task with a precise interface: a function `predict([subject, item], labeled)` must return $\Pr(\text{correct})$ for one subject–item pair, where `labeled` contains the labels already acquired on the same subject–benchmark pair at budgets $n \in \{0, 1, 3, 7, 15, 31\}$. Pairs are scored by the Brier score [Brier, 1950] at each budget and aggregated into a Brier ALC with weights $(0.1, 0.2, 0.2, 0.2, 0.2, 0.1)$—the trapezoidal area under the Brier-versus-$\log_2(n{+}1)$ curve.

Two facts about this metric shaped our design. First, 60% of the weight sits at $n \leq 7$, where only the pair-level success rate can be estimated with any precision; item-level discrimination can pay off only at $n = 15$ and $n = 31$, which together carry 30%. Second, the organisers' empirical-mean baseline is *worse* than the constant 0.5 at $n = 1$ (its prediction is 0 or 1), so the single most valuable ingredient is a well-shrunk estimate of the pair rate. Our predictor therefore *shrinks first*: an attribute-based prior at $n = 0$, a beta-binomial posterior mean of the pair rate for $n \geq 1$ [Efron and Morris, 1975], and only then item-level adaptation, which is *gated* by a leave-one-out (LOO) test on the acquired labels so that it is applied only when the pair's own labels show that it helps.

We also faced a constraint that defines the submission: the official training release (`aims-foundations/measurement-db`) is gated behind a Hugging Face login, and for most of the project the machine used for this work had no credentials (we did not create any account; the author's login arrived two days before the deadline). The predictor therefore contains *no parameter fitted on any response data*: every constant is a documented prior (Section 4), and the report tests their sensitivity, including for frontier-like subjects. Section 3 describes what the release was then used for: validating the shipped rule on real pairs, a cross-validated fit of the prior that lost to the hand-set one, and a pre-registered item-text difficulty prior that did not transfer to held-out benchmarks; neither was shipped. We see this as a strong, fully reproducible baseline for the competition and as a measurement of how far test-time adaptation alone can go.

Contributions. (1) A standard-library-only predictor and streaming acquisition policy that pass the official validator and run at hundreds of calls per second. (2) A local reproduction of the evaluation protocol on a *real* testbed of 17 pairs built with local open models and on a synthetic IRT testbed with known item probabilities, plus a validation of the shipped rule on 16,303 pairs of the official training release. (3) A paired-bootstrap analysis of what helps (prior, shrinkage), what helps marginally (item-level components) and what does not (sentence embeddings, uncertainty sampling, a fitted attribute prior, and a pre-registered item-text difficulty prior trained on 144 other benchmarks), with oracle bounds that locate the remaining headroom.

2 Task, interface and metric

A *subject* is a dictionary of strings (`normalized_name`, `provider`, `release_date`, `access_date`, `harness`, `harness_version`, `reasoning_effort`, `subject_features_extra`); an *item* has `item_content` (the task text), `item_features` (e.g. `tier=medium;points=1`), `interactors`, and an anonymous `benchmark_id` shared by all items of a benchmark. Missing values are empty strings. For every subject–benchmark pair with at least 80 distinct items, the evaluator splits the items 50/50 into an acquisition pool (never scored) and an evaluation pool. Acquisition candidates arrive in a fixed random order; an optional `acquisition_function(input, prediction, labeled, context)` returns `True` to reveal a label (31 at most), otherwise labels are sampled uniformly. The nested label sets after 0, 1, 3, 7, 15 and 31 reveals are passed to `predict` for every evaluation item, giving Brier scores $B_0, \ldots, B_{31}$ and

$$\text{ALC} = 0.1\,B_0 + 0.2\,B_1 + 0.2\,B_3 + 0.2\,B_7 + 0.2\,B_{15} + 0.1\,B_{31}, \tag{1}$$

averaged with equal weight over pairs (lower is better). Expected calibration error (ECE, ten equal-width bins) is reported as a diagnostic [Guo et al., 2017]. Test benchmarks are disjoint from training benchmarks; the evaluation runs offline with 8 hours per run.

3 Data

What we could and could not use. The official training data (`measurement-db`: item-level responses of many subjects on curated benchmarks) require accepting terms with a Hugging Face account. For most of the project we had no credentials and did not try to obtain the data in any other way; the predictor and all its constants were fixed before any of it was seen. Its public file listing (2026-10-22, revision `54074453da`) held 281 benchmark directories: 14.8 GB of `benchmarks`, `subjects`, `items` and `responses` tables plus 107.8 GB of traces and 154.7 GB of assets. On that day the author's login became available and we downloaded the 163 competition-eligible directories (binary, item-level responses; the eligibility flags live inside the `benchmarks` tables) without traces or assets, and used them as described next. We also inspected the organisers' inventory of eligible benchmarks (1,261 entries) but fitted nothing on any of them. The two local testbeds below only evaluate design choices; the few constants they influenced are marked in Section 4 and probed in Section 7.

The training release: validation and a fit that was not shipped. `fit_prior_from_measurement_db.py` builds one row per (subject, test condition, item) with the mean response over trials as label—16,718 pairs with at least 20 distinct items from 144 benchmarks, 3,118,742 rows (pairs capped at 200 items; 16,303 pairs have the 80 items the protocol requires)—and does two things (Appendix C). First, it evaluates the shipped prior-plus-shrinkage rule with the protocol in closed form (50/50 split, 31 random acquisitions, Eq. 1; without the item-level components, which change the ALC by < 0.001 locally): ALC 0.2051 [0.2042, 0.2060] against 0.2396 [0.2380, 0.2411] for the organisers' empirical mean and 0.2470 for the constant, with $\alpha_0 = 4$ the best of $\{2, 4, 8\}$. The ordering and the size of the gain measured locally thus transfer to real systems on real benchmarks. Second, it fits an L_2-penalised logistic regression on exactly the feature terms of the hand-set prior plus a provider table and benchmark fixed effects (nuisance parameters; the shipped intercept is their mean), pairs weighted equally, with five-fold cross-validation over whole benchmarks. The fitted prior *loses* to the hand-set one on held-out benchmarks at every penalty (budget-0 Brier 0.2421–0.2455 against 0.2388; paired differences at the selected setting +0.0033 [+0.0030, +0.0036] at budget 0 and +0.0009 [+0.0008, +0.0009] in ALC), and an intercept-only prior is worse still (0.2519). The ship-only-if-better rule therefore kept the hand-set constants; *no coefficient in the submission derives from the release*, and its only influence on the shipped predictor is this decision. Where the data speak, the fitted signs agree with the hand-set values; what held-out benchmarks punish is the provider table (Appendix C).

An item-text difficulty prior (pre-registered; not shipped). Benchmark difficulty dominates the pair rate (benchmark means have standard deviation 0.223) and the attribute prior cannot see it. We therefore tested, with design and ship rule fixed in `PREREG.md` (SHA-256 `bb54af425d...`) before any fit, whether the item text predicts difficulty on *unseen* benchmarks: a ridge regression on 262,144 signed-hashed buckets (words, bigrams, character trigrams, `item_features`) and 16 text statistics (length, LaTeX, code, answer choices, ...), trained on the item logit the hand-set prior leaves unexplained (109,233 items, benchmarks weighted equally) and stacked onto the prior as $T = Ws + B$ with W, B fitted on out-of-fold scores; shrinkage is unchanged. On the same benchmark-held-out folds, every choice nested inside the training benchmarks, the ALC changes by -0.0000 [-0.0013, +0.0014] (cluster bootstrap over 135 benchmarks), closing -0.1%, 0.1% and 0.1% of the gap to the pair-rate oracle at $n = 0, 1, 3$. The inner CV chose the largest penalty of the grid in 5 of 5 folds and the out-of-fold slope W was *negative* in all 5 (-0.32 to -0.03): text patterns learned on some benchmarks point the wrong way on others. Exploratory variants registered afterwards (penalties up to 10^4; statistics alone) give +0.0002 and +0.0001, the local testbeds +0.0019 (real, not significant) and +0.0003 (synthetic, not significant). Headroom exists—knowing each held-out benchmark's mean rate would gain -0.0061 ALC—but item text in this form does not reach it; the rule fails and the submission is unchanged (Appendix D).

Real Ollama testbed. To obtain item-level correctness labels of real language models we ran four small open-weight models locally with Ollama (Q4_K_M quantisation, greedy decoding, CPU only): Qwen2.5-Instruct 1.5B, 3B and 7B and Llama-3.2-3B-Instruct. Three public benchmarks were used, none of which appears in the PAIEC inventory (only a derived variant, `mr_gsm8k`, does): the first 200 GSM8K test items, the first 200 SVAMP test items (with the SVAMP problem type as `item_features`), and a 300-item mix of ten BIG-Bench-Hard subtasks (30 items each, subtask

name as `item_features`). Two harnesses are represented: a direct-answer prompt ("reply with only the final answer"; exact match after numeric or option normalisation) and, for GSM8K/SVAMP, an earlier 8-shot chain-of-thought run of the Qwen models whose greedy answers we re-used. Each (model, harness, benchmark) triple with at least 80 items is one pair, giving 17 pairs, 3,800 labelled items, and success rates from 8.0% to 79.0% (Table 7 in the appendix). The subject dictionaries carry the real model names, providers, release dates and harness names, so the budget-0 prior sees realistic inputs; the raw records are released with the code.

Synthetic IRT testbed. Because the real testbed is small (small models, three benchmarks), we complement it with a generator that follows a two-parameter item response model [Lord, 1980, Embretson and Reise, 2000]. Subject s has ability θ_s correlated with its visible attributes (release year, reasoning effort, parameter count) plus noise; benchmark b has a location μ_b, a discrimination a_b and $K = 5$ topic clusters with difficulty offsets $c_k \sim \mathcal{N}(0, \sigma^2_{\text{topic}})$; item i in cluster $k(i)$ has difficulty $b_i = \mu_b + c_{k(i)} + \gamma\, z_{\text{len}}(i) + \varepsilon_i$ and $\Pr(y_{si} = 1) = \sigma(a_b(\theta_s - b_i))$. Item texts mix cluster-specific vocabulary (35% of tokens), benchmark template words, ambient vocabulary and numbers, so that lexical similarity carries difficulty information exactly when $\sigma_{\text{topic}} > 0$; half of the benchmarks expose a `tier=easy/medium/hard` feature derived from difficulty terciles. The main setting uses 10 subjects $\times$ 16 benchmarks $= 160$ pairs with 100–240 items each, $\sigma_{\text{topic}} = 0.70$ and $\gamma = 0.3$; a sweep varies both. Because the true probabilities are known, we can report two oracle references: the Brier score of the true item probabilities (the floor of any predictor) and that of the evaluation-set success rate used as a constant (the floor of any predictor without item-level information).

4 Method

All constants below are hand-set and shipped in `model.py`; the few whose values were adjusted after looking at the local testbeds are marked † and have sensitivity analyses in Section 7.

Budget-0 prior. A logistic prior $p_0 = \sigma(z)$ with $z = \text{clip}(z_{\text{subj}} + z_{\text{item}}, \pm 1.5)$, so $p_0 \in [0.18, 0.82]$. The subject part is $z_{\text{subj}} = 0.7 \cdot (0.35\,\text{clip}(t - 2024.5, \pm 2) + e + 0.25\,\text{clip}(\log_2(N/30\text{B}), \pm 4) + 0.1\,\mathbb{1}[\text{non-default harness}])$, where t is the release year (fractional), $e \in \{0, 0.15, 0.3, 0.45\}$ encodes an empty, low, other or high `reasoning_effort`, and N is a parameter count parsed from the name ("7b", "70B"); when no count is present, name tokens such as *mini/nano/flash* contribute -0.3 and *pro/opus/ultra* contribute 0.2. An intercept (0) and a per-provider table (0 entries) complete the subject part; both are empty in the shipped predictor and are the slots a fitted coefficient table would fill. The item part parses `key=value` pairs in `item_features`: a tier/difficulty/level value matching *hard/expert/…* adds -0.6, *easy/basic/…* adds 0.5, a numeric level $\ell \in [1, 5]$ adds $-0.25(\ell - 3)$, and points above one subtract 0.1 each (capped at five); finally the item length adds $-0.12\,\text{clip}(\log_2(L/600), \pm 3)$ for L characters. The directions encode common knowledge (newer, larger, reasoning-enabled subjects do better; harder, longer, higher-value items are failed more often); the magnitudes were chosen to move the prior by at most a few tenths of a logit per attribute and were not tuned on either testbed.

Pair-level shrinkage. Given n acquired labels $y_1, \ldots, y_n$ of the same pair with item logits $z^{(j)}_{\text{item}}$ and mean $\bar{z}$, the prior rate is $r = \sigma(\text{clip}(z_{\text{subj}} + \bar{z}))$ and the posterior mean of a Beta–binomial model with prior strength $\alpha_0 = 4^\dagger$ pseudo-counts is $\rho = (\alpha_0 r + \sum_j y_j)/(\alpha_0 + n)$ [Efron and Morris, 1975]. The pair estimate for a target item is $p_{\text{pair}} = \sigma(\text{logit}\,\rho + z_{\text{item}} - \bar{z})$, i.e. the item cues shift the pair rate on the logit scale.

Item-level component 1: ridge features. From $n \geq 7$ labels, three cheap features of the item text—log length, digit density and log(1+number of sentences)—are standardised over the acquired items and a logistic regression with offset $\text{logit}\,p_{\text{pair}}$ and ridge penalty $\lambda = 4$ is fitted by six Newton steps; its adjustment $\text{clip}(\beta^\top \tilde{f}, \pm 1)$ is added to the target's logit.

Item-level component 2: TF-IDF nearest neighbours. Item texts (content, features and interactors, truncated to 4000 characters) are tokenised into lower-case words and numbers; the acquired set defines the vocabulary and document frequencies, with sublinear term frequency, smoothed idf and ℓ_2 normalisation (bigram weight 1; target terms absent from the acquired vocabulary are dropped:

no). With cosine similarities s_j between the target and acquired item j and weights $w_j = (\kappa s_j)^2$, the correction is a shrunken kernel-weighted residual,

$$p = \mathrm{clip}\Big(a + \frac{n}{n + N_0} \cdot \frac{\sum_j w_j\,(y_j - a)}{\tau + \sum_j w_j}\Big), \qquad N_0 = 4,\ \tau = 2, \tag{2}$$

where the anchor a is the pair estimate (optionally including the ridge adjustment) and $\kappa = 2$ by default. With at most 31 short documents most bigrams are unique, so cosine similarities are small (typically below 0.15 even for related items), which deliberately keeps the correction small unless items share many terms; Appendix A compares a sharper unigram-only representation.

Leave-one-out gating. From $n \geq 5$ labels, every configuration in $\{\text{ridge off, on}\} \times \{\kappa \in \{0, 1, 2, 4\}\}$ ($\kappa = 0$ means no kNN) is scored by its leave-one-out Brier score on the acquired labels themselves: label j is predicted from the other $n - 1$ labels, including a leave-one-out shrinkage estimate and a leave-one-out ridge fit. The best configuration replaces plain shrinkage only if its mean per-item improvement exceeds 2 standard error(s) of that mean[†]; otherwise the predictor returns p_{pair}. This makes adaptation *opt-in per pair and budget*: a pair whose labels show no exploitable structure is predicted by shrinkage alone. Below 5 labels no item-level component is used at all (in preliminary runs an always-on kNN from one or three labels only added noise at B_1 and B_3).

Acquisition policy. `labeling.py` implements the streaming interface and never raises. Candidates are accepted in stream order (uniformly random, hence the same label distribution as the evaluator's default) except that a candidate whose maximum TF-IDF cosine similarity to an already acquired item exceeds 0.6 is skipped while fewer than 90% of the remaining candidates are needed; it always accepts when the remaining candidates are needed to spend the budget. We compare this *novelty* filter with the evaluator's random policy, plain stream order, the organisers' uncertainty band $|p - 0.5| \leq 0.15$ [Truong et al., 2026], and a novelty-plus-uncertainty variant (skip $|p - 0.5| > 0.45$ once 7 labels are known).

5 Implementation

Predictions are clipped to $[0.01, 0.99]$; `predict` is wrapped in a fallback chain (full model → shrinkage only → smoothed label mean → 0.5), accepts labels given as booleans, numbers or strings, tolerates missing fields and non-dictionary inputs, and is deterministic. The submission ZIP (11.7 kB; members `README.md`, `labeling.py`, `model.py`) contains no `models.txt` and no `requirements.txt`: `model.py` imports only `math` and `re`, and `labeling.py` additionally `importlib`, `os` and `sys` to load its sibling `model.py` by path; it loads no files and makes no network calls. The organisers' `check_submission_zip.py` (commit `cc88a959`) passes on the ZIP; our smoke test calls `predict` with the organisers' sample inputs at every budget, with malformed labels and inputs (42 cases) and checks determinism. Mean latency is 1.92 ms per call on short items and 0.99 ms at budget 31 with 300-word items (maximum 27.0 ms), about 1,010 calls per second on one laptop core.

6 Experimental setup

Our harness reproduces the protocol of Section 2: items of each pair are split 50/50 with a seeded shuffle, the acquisition pool is streamed in a seeded random order, the policy under test selects up to 31 labels, and the nested prefixes of that one trajectory are passed to every predictor for every evaluation item, so all methods are compared on identical labels. The evaluator's default policy is reproduced as accept-with-probability labels_remaining/items_remaining. We report Brier ALC with 95% bootstrap confidence intervals over pairs (1,000 resamples), paired bootstrap differences between methods (same pairs, same trajectories), per-budget Brier scores and pooled ECE. Baselines: the constant 0.5; the organisers' empirical mean (their code: 0.5 without matching labels, the raw mean otherwise); our prior alone; shrinkage alone; shrinkage with each item-level component always on; the shipped LOO-gated model.

Table 1: Brier ALC (lower is better) with 95% bootstrap CIs over pairs and pooled ECE on the two local testbeds, all methods evaluated on identical random-policy acquisition trajectories. Oracles are references, not predictors.

	Synthetic IRT (160 pairs)		Real Ollama (17 pairs)	
Method	ALC [95% CI]	ECE	ALC [95% CI]	ECE
constant 0.5	0.2500 [0.2500, 0.2500]	0.111	0.2500 [0.2500, 0.2500]	0.127
empirical mean (organisers)	0.2208 [0.2055, 0.2362]	0.114	0.2871 [0.2369, 0.3303]	0.170
attribute prior only	0.2148 [0.2040, 0.2251]	0.041	0.2340 [0.2198, 0.2468]	0.076
prior + shrinkage	0.1797 [0.1691, 0.1900]	0.035	0.2228 [0.1951, 0.2460]	0.063
shrinkage + ridge (always on)	0.1814 [0.1708, 0.1918]	0.029	0.2199 [0.1945, 0.2416]	0.061
shrinkage + kNN (always on)	0.1794 [0.1687, 0.1895]	0.033	0.2226 [0.1949, 0.2453]	0.061
full: LOO-gated ridge + kNN (shipped)	0.1797 [0.1691, 0.1900]	0.032	0.2224 [0.1949, 0.2454]	0.061
oracle: pair success rate	0.1598 [0.1483, 0.1710]	0.000	0.1961 [0.1675, 0.2208]	0.000
oracle: true item probability	0.1379 [0.1280, 0.1482]	0.007	—	—

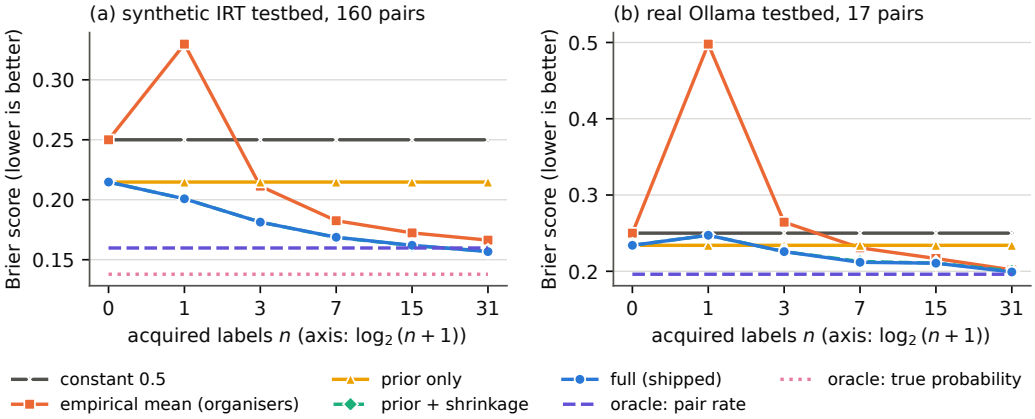


Figure 1: Brier score per budget (the ALC of Eq. 1 is the trapezoidal area under these curves on the $\log_2(n+1)$ axis). The empirical mean is worse than the constant at $n = 1$; shrinkage removes that penalty; the full model separates from shrinkage only at $n \geq 15$. Dashed/dotted lines are the oracle references.

7 Results

Main comparison. Table 1 and Figure 1 give the headline numbers. On the real testbed the shipped predictor reaches 0.2224 [0.1949, 0.2454] against 0.2871 [0.2369, 0.3303] for the empirical mean and 0.25 for the constant; the paired difference to the empirical mean is -0.0647 [-0.0872, -0.0415] (significant; the full model is better on 82% of pairs) and to the constant -0.0276 [-0.0506, -0.0050] (significant). On the synthetic testbed the corresponding ALCs are 0.1797 [0.1691, 0.1900] versus 0.2208 and 0.25, with paired differences -0.0411 [-0.0488, -0.0339] and -0.0703 [-0.0805, -0.0592] (both significant). The empirical mean is *worse* than the constant at $n = 1$ on both testbeds ($B_1 = 0.330$ and 0.498) and only overtakes it from $n = 3$ (synthetic) and $n = 7$ (real); on the real testbed its ALC (0.2871) is *above* the constant's 0.25 (paired difference +0.0371 [-0.0081, +0.0779], not significant).

Where the gain comes from. Decomposing the ablation rows of Table 1: the prior alone improves on the constant by -0.0352 (synthetic) and -0.0160 (real) ALC; shrinkage adds -0.0351 [-0.0415, -0.0285] and -0.0113 [-0.0283, +0.0042] on top (the real-testbed difference is not significant with 17 pairs). The item-level components are a much smaller effect. Against shrinkage, always-on kNN changes the ALC by -0.0003 (synthetic) and -0.0002 (real), always-on ridge by +0.0017 and -0.0029, and the LOO-gated full model by +0.0000 [-0.0002, +0.0003] (not significant) and -0.0004 [-0.0012, +0.0003] (not significant); at $n = 31$ alone the gated model changes the Brier score by -0.000 and -0.001. Replacing TF-IDF by `sentence-transformers/all-MiniLM-L6-v2` embeddings

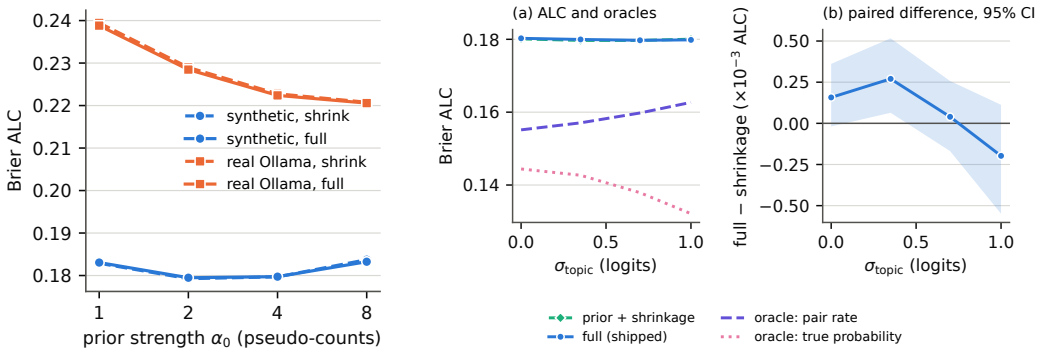


Figure 2: Left: Brier ALC vs. prior strength α_0 (dashed: shrinkage, solid: full model). Right: synthetic sweep of the topic-difficulty spread σ_{topic} at $\gamma = 0.3$: (a) ALC of shrinkage, the full model and the two oracles; (b) paired difference full − shrinkage with its 95% CI over pairs (negative favours the full model).

changes the full model's ALC by -0.0001 [-0.0005, +0.0001] (not significant), so we ship TF-IDF; Appendix A adds the LOO gate against always-on components and a sharper unigram TF-IDF, which makes the kNN worse. We report these as what they are: item-level adaptation from at most 31 labels is a marginal gain whose sign has to be protected by the gate.

Headroom. The oracle rows explain why. On the synthetic testbed, knowing the true probability of every item gives an ALC of 0.1379, knowing only the evaluation-set success rate 0.1598: perfect item-level discrimination is worth 0.0219 ALC, to be recovered from 15–31 labels at budgets that carry 30% of the weight. Shrinkage (0.1797) is 0.0199 above the rate oracle; that gap is the price of estimating a rate from few labels, concentrated at $n \leq 3$ (B_1: 0.041; B_{31}: -0.003), and only a better prior, i.e. training data, can reduce it. On the real testbed the gap is 0.0266 (0.2228 vs. 0.1961). The two parts of the headroom are of similar size but call for different remedies: a prior fitted to real subjects and benchmarks, and item difficulty information from outside the pair.

Prior strength and text signal. Figure 2 gives the ALC for $\alpha_0 \in \{1, 2, 4, 8\}$ (shrinkage is best at 2 on synthetic and 8 on real; full-model range 0.0038 and 0.0182; we ship $\alpha_0 = 4$, the smallest worst-case regret and the best of $\{2, 4, 8\}$ on the training release) and the synthetic sweep over σ_{topic}, where the item-level effect stays two orders of magnitude below the item-level headroom (0.0080 to 0.0306): the gate protects against harm, it cannot create information.

Acquisition policies. None of the five policies we compared is significantly different from the evaluator's random policy on either testbed (Appendix A, Table 6): the shipped novelty filter gives -0.0013 [-0.0038, +0.0013] (synthetic) and -0.0044 [-0.0108, +0.0025] (real), the organisers' uncertainty band +0.0001 and -0.0041. Selecting by $|p - 0.5|$ distorts the pair-rate estimate that carries most of the metric; we ship the policy closest to random that removes near-duplicates.

Frontier-like subjects. The official test set has subjects our testbeds lack (2025–2026 releases, long reasoning efforts, very large models, agent harnesses); over 160 such descriptions the budget-0 prior ranges from 0.39 to 0.82 (the cap; Appendix E). What a wrong prior costs is exact under shrinkage (Table 4): a prior of 0.7 on a pair whose true rate is 0.3 costs 0.062 ALC and loses to the constant (excess 0.040), whereas shrinkage beats the constant whenever $|r - 0.5| \geq 0.2$ and $|p_0 - r| \leq 0.3$. On the real testbed an optimistic shift of every subject's prior logit costs significantly ($+0.5$: +0.0134), a pessimistic one does not (-0.5: -0.0034, not significant); on the training release the hand-set prior averages 0.510 against a mean pair rate of 0.537, so the intercept stays at zero.

Replication and calibration. Appendix B reports replication over generator seeds and acquisition trajectories (the full model beats the empirical mean significantly in 4 of 4 further real trajectories). Pooled ECE (Table 1) is 0.032/0.061 (synthetic/real) for the full model against 0.114/0.170 for the empirical mean and 0.111/0.127 for the constant: shrinkage towards a sensible prior also fixes calibration at small budgets.

8 Limitations

Local-versus-official gap. Our real testbed covers four small open models on three benchmarks with two harnesses; the official test set contains frontier systems on many held-out benchmarks. In particular the subject part of the prior (release date, reasoning effort, size) has essentially no variance on our testbed and is validated only on the synthetic generator, whose attribute–ability relation we chose ourselves. Absolute ALC values will differ; we expect the *ordering* shrinkage $>$ empirical mean $>$ constant at small budgets to transfer because it follows from the metric, but the sign and size of the item-level effects are testbed-dependent. *A fitted prior did not help.* On the training release a logistic prior over the same visible attributes lost to the hand-set one under benchmark-held-out cross-validation, and an intercept-only prior lost by more: across benchmarks the pair success rate is dominated by benchmark difficulty, which the interface hides for test benchmarks, so visible subject attributes buy little at budget 0 and the gain over the empirical mean comes from shrinkage. Richer priors need information about the test benchmark itself. The most obvious source, the item text, did not provide it in our pre-registered test (hashed n-grams and text statistics, Section 3); pretrained text encoders remain untested, and labels of other subjects on the same benchmark are outside the stateless interface. *Small real testbed.* 17 pairs give wide confidence intervals; non-significant differences are stated as such. *Stateless by design.* Labels are never pooled across subjects of the same benchmark (possible via `benchmark_id`, but outside the spirit of the task). *Weak text signal.* With at most 31 labels, lexical similarity recovers little of the item-level headroom on either testbed. *Acquisition filter without measurable effect.* The novelty filter fires on templated real-testbed items (Appendix A) but changes the ALC by less than 10^{-4} relative to stream order; its value on heavily templated benchmarks is untested.

9 Reproducibility

All code, results and this report are released at `https://github.com/neelsirivara/neurips2026-paiec` (also attached to the OpenReview submission). `experiments/src/model.py` and `labeling.py` are the shipped files; `build_submission.py` creates the ZIP and runs the organisers' validator; `smoke_test.py` checks interface and timing; `real_testbed_generate.py` regenerates the Ollama testbed; `simulate.py` is the IRT generator and `harness.py` the protocol; `run_experiments.py -suite all`, `run_extras.py`, `prior_sensitivity.py`, `novelty_stats.py`, `fit_prior_from_measurement_db.py` and `text_prior.py` (round 3; its pre-registration and addendum are in `experiments/results/round3/`) write every result JSON in `experiments/results/`; `build_paper.sh` runs `make_macros.py`, `make_figures.py` and `make_audit.py` (which re-derives every directional claim of this report from the JSON files) and compiles the PDF, so no number is typed by hand. All seeds are fixed. Hardware: one laptop CPU (8 cores, 34 GB RAM, no GPU).

10 Disclosure

Training data: the predictor has no fitted parameters. `aims-foundations/measurement-db` (revision `54074453da`; the `subjects`, `items` and `responses` tables of its 163 competition-eligible benchmark directories, 144 of which yielded usable pairs) was downloaded on 2026-10-22 with the author's account and used only (i) to evaluate the already-fixed shipped rule, (ii) to fit a candidate attribute prior and (iii) to train and cross-validate a candidate item-text prior on the `content` and `item_features` of 515,067 items, neither of which was shipped (Section 3; the 144 benchmarks are listed in Appendix D); its traces and assets were not downloaded. No other source from the inventory was used. *Evaluation data:* GSM8K and SVAMP test sets and ten BIG-Bench-Hard subtasks (local copies of the public Hugging Face datasets), used only to generate correctness labels of local models. *Pretrained models:* Qwen2.5-Instruct 1.5B/3B/7B and Llama-3.2-3B-Instruct via Ollama, as evaluation subjects only; `sentence-transformers/all-MiniLM-L6-v2` in one offline ablation, not shipped. No commercial API was called. *Compute:* a CPU-only laptop (8 cores, 34 GB RAM); about three hours of testbed generation, one hour for the experiment suite, one hour for the release fit, under an hour for the item-text prior. *LLM coding assistant:* code, experiments and text were drafted by Anthropic's Claude (Claude Code) running autonomously on the author's machine; the author reviewed the pipeline and is responsible for the content.

References

Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975. doi: 10.1080/01621459.1975.10479864.

Susan E. Embretson and Steven P. Reise. *Item Response Theory for Psychologists*. Psychology Press, 2000. doi: 10.4324/9781410605269.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1706.04599.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. arXiv:2211.09110.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Routledge, 1980. doi: 10.4324/9780203056615.

Lorenzo Pacchiardi, Lucy G. Cheke, and José Hernández-Orallo. 100 instances is all you need: Predicting the success of a new LLM on unseen data by testing on a few instances, 2024. arXiv:2409.03563; presented at the KDD 2024 Workshop on Evaluation and Trustworthiness of Generative AI Models.

Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking of language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2308.11696.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.

Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.10938.

Stanford AIMS Lab. Predictive AI evaluation competition (NeurIPS 2026). Competition web page, 2026. URL `https://aims.stanford.edu/competition`. Accessed 2026-10-21.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL `https://github.com/aims-foundations/paiec_baseline`.

A Additional ablations

Prior strength. Table 10 gives the ALC for $\alpha_0 \in \{1, 2, 4, 8\}$. The best value differs between testbeds (shrinkage: 2 on synthetic, 8 on real), reflecting the wider spread of pair rates in the synthetic generator; the ALC of the full model varies by 0.0038 (synthetic) and 0.0182 (real) over the range, so the choice matters at the third decimal. On the training release $\alpha_0 = 4$ gives 0.2051 against 0.2052 (2) and 0.2086 (8).

When does text help? The synthetic sweep (Figure 2, right; Table 9) varies how much difficulty is explained by topic. With no topic or length signal ($\sigma_{\text{topic}} = 0.00$, $\gamma = 0.0$) the full model's paired difference to shrinkage is +0.0002 [-0.0000, +0.0005], the gate's cost when there is nothing to find; at the strongest setting ($\sigma_{\text{topic}} = 1.00$) it is -0.0002 [-0.0005, +0.0001], while the item-level headroom (rate oracle minus true-probability oracle) grows from 0.0080 to 0.0306.

Gating. Replacing the LOO gate by always-on components at their default settings gives 0.1808 versus 0.1797 (synthetic; paired gated−always -0.0011 [-0.0017, -0.0005], significant) and 0.2203 versus 0.2224 (real; +0.0021 [-0.0009, +0.0048], not significant). The gate pays where always-on components overfit (the synthetic ridge) and costs a little where a component carries real signal (on the real testbed longer GSM8K problems are failed more often, which the always-on ridge exploits at $n \geq 15$); the second effect is within noise. Requiring 0, 1 or 2 standard errors of LOO improvement gives full-model ALCs of 0.1808, 0.1799 and 0.1797 (synthetic) and 0.2219, 0.2223 and 0.2224 (real); we ship the most conservative setting.

Sentence embeddings. The embeddings were computed offline for the real testbed (the model is on the validator's allow-list via `models.txt`, so it could have been shipped). They change the full model's ALC from 0.2224 to 0.2223 (paired -0.0001 [-0.0005, +0.0001], not significant) and the always-on kNN variant from 0.2226 to 0.2252 (+0.0026 [+0.0000, +0.0055], significant), at the cost of a neural dependency, model loading and about 10 ms per embedding on CPU.

Sharper lexical similarity does not help. A unigram-only TF-IDF whose target terms are restricted to the acquired vocabulary separates same-topic from other-topic synthetic items far better (mean cosine about 0.3 versus 0.1 in the generator) but makes the always-on kNN *worse*: paired against the shipped representation, the always-on kNN changes by +0.0009 [-0.0001, +0.0020] (synthetic) and +0.0026 [+0.0006, +0.0050] (real), and the gated full model by +0.0004 [+0.0001, +0.0008] and +0.0006 [-0.0002, +0.0018]. With a handful of neighbours per topic the kernel-weighted residual is dominated by label noise; the bigram-diluted similarity acts as a regulariser. Item-level signal exists (the oracles show it) but 31 labels are too few to exploit it through similarity alone.

Acquisition policies. Table 6 compares policies for the full predictor on identical pools and streams. Plain stream order differs from the evaluator's random policy by -0.0013 [-0.0038, +0.0013] (synthetic, not significant) and -0.0044 [-0.0107, +0.0019] (real, not significant)—both are uniform samples, so this is the trajectory-to-trajectory noise floor of the comparison. The shipped novelty filter gives -0.0013 [-0.0038, +0.0013] and -0.0044 [-0.0108, +0.0025]. It does fire on the real testbed: on 10 of 17 pairs (40 candidates skipped in total, at most 10 per pair; templated BIG-Bench-Hard items such as *navigate* and *boolean expressions*, and SVAMP word problems sharing a template, reach cosine 0.98), and on none of the 160 synthetic pairs, whose pseudo-word items never get that similar—yet its ALC equals that of plain stream order to four decimals (0.2180 vs. 0.2180): skipping a near-duplicate changes which uniformly random items are labelled, not their distribution. The organisers' uncertainty band gives +0.0001 [-0.0027, +0.0030] and -0.0041 [-0.0096, +0.0016], and the novelty-plus-uncertainty variant -0.0013 and -0.0044: selecting by $|p - 0.5|$ reorders a near-random sample at small budgets and, once item-level corrections exist, biases it towards items of intermediate predicted difficulty, distorting the pair-rate estimate that carries most of the metric. None of the policies is significantly different from random on either testbed; we ship the one that is closest to random while removing redundant items.

B Replication

Regenerating the synthetic testbed with two further seeds gives full-model ALCs of 0.2266 and 0.1963 (empirical mean 0.2737 and 0.2438; shrinkage 0.2264 and 0.1963). On the real testbed, four further acquisition trajectories give full-model ALCs between 0.2127 and 0.2240 (empirical mean 0.2511–0.2841), and the full model beats the empirical mean significantly in 4 of 4 trajectories. A 3×3 grid over the kNN scale and anchor ($\kappa \in \{1, 2, 4\}$, $\tau \in \{1, 2, 4\}$) spans 0.1806–0.1809 on a smaller synthetic generator, i.e. the kernel constants matter little.

C Fitting pipeline details

`fit_prior_from_measurement_db.py` runs in five steps, each a sub-command. *plan*: list the repository, download the small `benchmarks` table of every directory and keep those with `response_type = binary` and `granularity = item` (the organisers' eligibility rule in `tools/prepare_data.py`); record the bytes still to download and stop above a cap (30 GB by default). *download*: fetch the eligible `subjects`, `items` and `responses` tables into the organisers' payload layout. *fit*: build one row per (subject, test condition, item) with the mean response over trials

as label (fractional labels enter as two weighted rows), each pair receiving total weight one; compute the subject and item terms with the shipped `model.subject_terms` and `model.item_terms`, so that fitted coefficients map one-to-one onto the constants of Section 4; add a one-hot for providers with at least 15 pairs and one for each training benchmark; fit an L_2-penalised logistic regression. Five-fold cross-validation holds out whole benchmarks; the penalty C and the logit cap are selected by the pair-weighted budget-0 Brier score of the prior on held-out pairs, and $\alpha_0 \in \{2, 4, 8\}$ by the closed-form shrinkage ALC of the protocol (50/50 split, 31 random acquisitions, Eq. 1) on held-out pairs with at least 80 items, paired against the hand-set prior on the same pairs. *testbeds*: evaluate the fitted table inside the harness on both local testbeds against the shipped constants. *dry-run*: generate tables with the release schema (shared pool of subjects, random subsets per benchmark, benchmark difficulty offsets with standard deviation one logit, labels from a known coefficient table over the prior's own terms, two ineligible benchmarks) and run *fit* on them. The dry run used 14 eligible benchmarks, 299 pairs and 47,784 rows; a light-regularisation fit recovers all 13 slope signs with maximum error 0.05 and mean error 0.02 logits, and the fitted benchmark effects have standard deviation 0.57. Its cross-validated selection chose $C = 0.003$ and $\alpha_0 = 4$ and did not beat the hand-set prior (budget-0 Brier +0.0036 [+0.0004, +0.0069]; shrinkage ALC +0.0013 [+0.0001, +0.0027]), so on these tables the rule would have kept the hand-set constants.

The run on the release. Revision `54074453da`; 163 eligible directories, 144 with at least one pair of 20 or more distinct items; the pair unit is (benchmark directory, subject, test condition), since the release records several test conditions per subject on many benchmarks; 16,718 pairs, 3,118,742 rows (each pair capped at 200 items); 22 providers with at least 15 pairs received a coefficient. Pair success rates have mean 0.537 and standard deviation 0.265 across pairs (benchmark means: standard deviation 0.223), while the hand-set p_0 has mean 0.510 and standard deviation 0.072. The penalty grid ran from $C = 0.001$ to 1; the selection took $C = 0.001$, cap 1.5 and $\alpha_0 = 2$ (cross-validated budget-0 Brier 0.2421, shrinkage ALC 0.2061). The fitted benchmark effects have standard deviation 0.46 logits. Fitted vs. effective hand-set coefficients: release year +0.14 vs. +0.24 per year, size +0.08 vs. +0.17 per doubling, hard-tier cue -0.65 vs. -0.60, easy-tier cue +0.08 vs. +0.50, item length +0.08 vs. +0.12; the reasoning-effort and harness terms shrink to about zero (-0.08, -0.02; 58% of pairs carry no provider field and most carry no effort or harness field), and the provider table spans -0.23 (`hugging face`) to +0.55 (`unknown`). Inside the harness on the local testbeds the fitted table changes the full model's ALC by +0.0021 [-0.0006, +0.0045] (real) and +0.0058 [+0.0032, +0.0081] (synthetic); the result files are `prior_fit.json` and `prior_fit_testbeds.json`, and the ship rule's verdict is recorded there (not shipped).

D Item-text difficulty prior: details

Pre-registration and outcome. `experiments/results/round3/PREREG.md` (SHA-256 prefix `bb54af425d`, recorded in every result file and verified unchanged at build time) fixed features, target, model, nested selection, evaluation and ship rule before any text model was fitted. `PREREG_ADDENDUM_1.md` (verified unchanged) registered the two exploratory variants after the primary folds had run, explicitly without influence on the decision. The rule: ship iff (a) the benchmark-held-out ALC difference (text minus hand-set) has a 95% cluster-bootstrap CI below zero, (b) the real testbed is not significantly worse, and the engineering gates hold (≥ 200 calls/s on one core with a new target item per call, weights ≤ 5 MB, validator and smoke test pass, never raises). Outcome: (a) fails, (b) holds, gates pass; the text prior is not shipped.

Features and target. JSON contents (the release stores many items as JSON objects) are flattened into `key: value` lines; words and word bigrams come from the first 2,000 characters (digit runs mapped to a length token), character trigrams from the first 500, and `item_features` tokens form a fourth family; each family is sublinear-tf weighted and ℓ_2-normalised, then hashed with `zlib.crc32` (sign from bit 31) into 262,144 buckets. The target of item i is $\mathrm{logit}\,\frac{n_i r_i + 0.5}{n_i + 1}$ minus the mean hand-set subject logit of the n_i pairs that evaluated it and minus its hand-set item logit, clipped to ± 5. At most 1,000 items per benchmark (109,233 in total) are used for fitting; all 515,067 items of the frame are scored (237,628,268 non-zeros). Exact duplicates of a test item's content in a training benchmark are rare (0.02% of frame items on average), so shared items do not inflate the cross-validation.

Nested selection and fits. In each outer fold a four-fold benchmark-grouped inner CV chooses the ridge penalty from $\{0.03, 0.1, 0.3, 1, 3, 10\}$ by weighted MSE of the target; the inner out-of-fold scores give W, B (pair-weighted logistic likelihood), the cap ($\{1.5, 2.5, 3.5\}$, training budget-0 Brier) and the within-pair weight λ of the text offset ($\{0, 0.5, 1\}$, training closed-form ALC). Selected: penalty 10 in every fold (the grid maximum), caps 1.5, 2.5, $\lambda = 0$, W from -0.32 to -0.03 and B from +0.06 to +0.21. The held-out R^2 of the residual target was -0.06 to 0.04 across folds (inner CV: -0.04 to -0.00). The final fit on all 144 benchmarks again selected penalty 10 with $W = -0.20$, $B = +0.14$, cap 1.5 and $\lambda = 0$; it is kept as `text_prior_candidate.json` (1.40 MB) and not shipped.

Table 2: Item-text prior: benchmark-held-out cross-validation on the training release (closed-form protocol, shrinkage with $\alpha_0 = 4$ unless stated, no item-level components). B_0 over all items of each of the 16,718 pairs; B_1–B_7 and ALC on the evaluation half of the 16,303 pairs with at least 80 items. Exploratory rows were registered after the primary result; the oracles are references.

Method	B_0	B_1	B_3	B_7	ALC
hand-set prior (shipped)	0.2388	0.2235	0.2083	0.1956	0.2051
hand-set + item-text prior (pre-registered)	0.2391	0.2234	0.2083	0.1956	0.2051
same, $\alpha_0 = 2$	0.2391	0.2240	0.2086	0.1955	0.2052
same, $\alpha_0 = 8$	0.2391	0.2278	0.2141	0.2002	0.2086
exploratory E1: penalty up to 10^4	0.2397	0.2239	0.2085	0.1957	0.2053
exploratory E2: 16 statistics only	0.2395	0.2237	0.2084	0.1956	0.2052
hand-set + fitted intercept shift only	0.2398	0.2240	0.2086	0.1957	0.2054
intercept only (training mean)	0.2519	0.2320	0.2127	0.1975	0.2096
oracle: held-out benchmark mean	0.2187	0.2115	0.2022	0.1932	0.1990
oracle: pair success rate	–	0.1756	0.1756	0.1756	–

Results. Table 2. Per budget the paired differences are -0.0000 [-0.0025, +0.0027] at $n = 1$ and -0.0000 [-0.0013, +0.0014] at $n = 3$ (gap to the pair-rate oracle closed: 0.1% [-5.6, 6.0] and 0.1% [-4.2, 4.3]); budget-0 Brier changes by +0.0003 [-0.0036, +0.0046]; the plain pair bootstrap gives -0.0000 [-0.0001, +0.0001]. The text prior has the lower mean ALC in 1 of 5 folds (significantly in 0) and on 57 of 135 benchmarks. Refitting only the intercept shift ($W = 0$; a descriptive decomposition) gives +0.0003 [-0.0007, +0.0009]: the hand-set prior's level is already right on held-out benchmarks. With the text prior, $\alpha_0 = 2$ gives +0.0001 and $\alpha_0 = 8$ +0.0035 (significant) relative to the hand-set prior at $\alpha_0 = 4$. The exploratory variants (E1: grid 10, 100, 1000, 10000, selected 100, 1000, 10000; E2: statistics only, grid 0.1, 1, 10, 100, selected 100) changed the ALC by +0.0002 [-0.0007, +0.0011] and +0.0001 [-0.0010, +0.0014] and again had negative W in 5 and 5 of 5 folds. The held-out benchmark-mean oracle closes 30%, 25% and 19% of the gap at $n = 0, 1, 3$.

Local testbeds. Full harness, identical trajectories: real ALC 0.2224 (hand-set) vs. 0.2243 (text), paired +0.0019 [-0.0002, +0.0037], text better on 35% of pairs; without the BIG-Bench-Hard pairs, whose items may overlap the `fewshot_ttt_bbh` training benchmark, +0.0017 [-0.0007, +0.0042] (13 pairs). Synthetic: 0.1797 vs. 0.1800 (+0.0003 [-0.0002, +0.0008]); its pseudo-word items carry no transferable text signal by construction.

Engineering gates and the shipped file. With the candidate loaded, `predict` runs at 3,056, 673 and 417 calls/s on one core with a new 15-, 300- and 2,000-word target item per call at budget 31 (the per-item text score is cached), the weights load in 22 ms, and the smoke test passes 44 cases, including malformed JSON contents and a corrupted weight specification (which falls back to the hand-set prior); standard-library and numpy scores agree to 9.0×10^{-9}. Since the rule failed, the submission ZIP is unchanged (SHA-256 prefix `3f0d15c879d9`); the development `model.py` keeps the text-prior hooks, which are inactive without a weight file and reproduce the shipped file's predictions bit for bit on 5,544 of 5,544 test cases.

Benchmarks of the training release used (144). `abstract_reason`, `adaptivestep`, `advprompter`, `aegis`, `afrieval`, `afrimedqa`, `agentic_review_perturbation`, `agi_elo`, `ai2d_test`, `aider`, `algotune`, `alignment_faking`, `alpha_sql`, `appworld`, `auditing_sabotage_bench`, `averimatec`, `babilong`, `bbq`, `beavertails`, `benger`, `bertaqa`, `bird_sql`, `brace`, `braveguard`, `bridging_gap`, `causal_axioms`, `cellverse`, `ceval`, `chi_bench`, `clasheval`, `cmmlu`, `critic_discernment_game`, `crow`, `cruxeval`, `dave`, `devbench`,

editbench, edu_circuit_hw, egoschema, ehrflowbench, elicitation_game, emoji_attack, engdesign, enginemt_qa, erbench, evouna, exploitgym, faithcot, felm, fewshot_ttt_bbh, financebench, finegrain_t2i, haiid, hallusionbench, harmbench, healthadminbench, helm_afr, helm_bbq, helm_thaiexam, hle, icpc2_code_selector, igakuqa119, ikp, interchangeable_token_embeddings, jailbreakbench, judgetuning, kernelbench, kmmlu, kormedmcqa, legal_rag_bench, legalbench, lingoly, liveaopsbench, llm_bp_tag, llm_uncertainty_bench, llmail_inject, llms_lean_formalization, mathconstraint, mathvista_mini, medhelm_ehr_sql, medhelm_head_qa, medhelm_med_mcqa, medhelm_med_qa, medhelm_medbullets, medhelm_medcalc_bench, medhelm_medec, medhelm_medhallu, medhelm_pubmed_qa, medhelm_race_based_med, mind2web, mj_bench, mmbench_v11, mmlu, mmlupro, monkey_power_laws, morphkv, multi_swebench, mypcbench, naep_llm_students, naturebench, nester, nyu_ctf_bench, ood_prediction, openbiorq, os_harm, perfcodebench, perumedqa, planbench, proofaug, prox, pxplore, real_webagents, realtimeqa, reliancescope, researchcodebench, revolve, risebench, robust_reasoning_benchmark, safeagentbench, sgrades, sib200, sketchjudge, software_world_models, statqa, subjective_qa, sugarcrepe, sustainable_food, swe_rebench, swe_smith, swe_together, swebench_live, swepolybench, tau2_bench, tensortrust, truthfulqa_mc, visual_memory, visual_riddles, visualwebarena, vl_rewardbench, weavebench, wikihow_agent, workarena, workbench_revisited, worldcentralbanks.

E Prior sensitivity for frontier-like subjects

Table 3 gives the budget-0 prior p_0 of the shipped constants for subject descriptions that the official test set is likely to contain and our testbeds do not. Table 4 gives the exact expected cost of a miscalibrated prior under the shipped shrinkage rule, and Table 5 the measured effect of shifting every subject's prior logit on the two testbeds (`PRIOR_INTERCEPT` in `model.py`; all other constants unchanged; identical trajectories; generated by `prior_sensitivity.py`).

Discussion. The official test set has subjects our testbeds lack (2025–2026 releases, long reasoning efforts, very large models, agent harnesses). Appendix E tabulates the budget-0 prior over 160 such descriptions: p_0 ranges from 0.39 to 0.82 (the cap); the organisers' GPT-4o and 2026 medium-effort sample subjects receive 0.49 and 0.66, our testbed models 0.36–0.39. What a wrong prior costs is exact: with $\alpha_0 = 4$ and n labels the expected Brier excess over the pair-rate oracle is $\mathbb{E}[(\rho_n - r)^2]$ with binomial successes (Table 4). A prior of 0.7 on a pair whose true rate is 0.3 costs 0.062 ALC and loses to the constant (excess 0.040); shrinkage beats the constant whenever $|r - 0.5| \geq 0.2$ and $|p_0 - r| \leq 0.3$, and never when $|r - 0.5| < 0.15$, where the constant is close to optimal. Shifting every subject's prior logit by ± 0.5 or ± 1 (Table 5) is asymmetric on the real testbed: $+0.5$ and $+1$ cost +0.0134 and +0.0353 ALC (both significant), whereas -0.5 changes it by -0.0034 [-0.0117, +0.0049] (not significant)—small models at 8.0–79.0% accuracy tolerate a pessimistic prior; on the synthetic testbed zero is best and both ± 0.5 shifts cost significantly (+0.0037, +0.0047). On the training release the hand-set prior averages 0.510 against a mean pair rate of 0.537, i.e. it is not optimistic for real systems. We keep the intercept at zero and name it as the first constant to change if the official formative score points the other way.

Table 3: Budget-0 prior p_0 (no item cues; 600-character item) by release year and `reasoning_effort` for a subject with no size or name cue on the default harness (left), and for a high-effort subject on a non-default harness with a *mini* name cue, a 70B or a 400B parameter count (right). The organisers' sample subjects receive 0.49 (GPT-4o, 2024-05) and 0.66 (2026-01, medium effort, non-default harness).

	default harness, no name cue				high effort, non-default harness		
Release year	none	low	medium	high	mini	70B	400B
2023	0.44	0.46	0.49	0.51	0.48	0.58	0.69
2024	0.50	0.52	0.55	0.58	0.54	0.64	0.74
2025	0.56	0.58	0.61	0.63	0.60	0.70	0.78
2026	0.62	0.64	0.67	0.69	0.66	0.75	0.82

F Validator and smoke-test output

The organisers' validator exits with code 0 and prints

Table 4: Exact expected Brier ALC excess over the pair-rate oracle $r(1-r)$ of the shipped shrinkage rule ($\alpha_0 = 4$, clipping to $[0.01, 0.99]$) when the prior rate is p_0 and the true pair rate is r; the number of successes among the n acquired labels is binomial(n, r) and the ALC weights are those of Eq. 1. The last column is the excess of the constant 0.5; † marks cells where the constant is better.

	prior rate p_0						
True rate r	0.3	0.4	0.5	0.6	0.7	0.8	constant 0.5
0.2	0.010	0.020	0.036	0.060	0.089	0.126†	0.090
0.3	0.009	0.012	0.022	0.039	0.062†	0.091†	0.040
0.4	0.013†	0.010†	0.013†	0.023†	0.040†	0.063†	0.010
0.5	0.024†	0.014†	0.011†	0.014†	0.024†	0.040†	0.000
0.6	0.040†	0.023†	0.013†	0.010†	0.013†	0.023†	0.010
0.7	0.062†	0.039	0.022	0.012	0.009	0.012	0.040
0.8	0.089	0.060	0.036	0.020	0.010	0.007	0.090

Table 5: Brier ALC of the prior alone, shrinkage and the full model when every subject's prior logit is shifted by `PRIOR_INTERCEPT` (random policy, same trajectories as Table 1). The shipped value is 0.

	Real Ollama (17 pairs)			Synthetic IRT (160 pairs)		
Shift (logits)	prior	shrinkage	full	prior	shrinkage	full
-1.0	0.2511	0.2229	0.2229	0.2689	0.1959	0.1955
-0.5	0.2319	0.2191	0.2190	0.2286	0.1835	0.1834
$+0.0$	0.2340	0.2228	0.2224	0.2148	0.1797	0.1797
$+0.5$	0.2663	0.2364	0.2358	0.2268	0.1844	0.1844
$+1.0$	0.3250	0.2585	0.2576	0.2527	0.1932	0.1932

```
OK: paiec_submission.zip looks like a valid submission ZIP.
```

(full output in `experiments/results/validator_output.txt`; ZIP SHA-256 prefix `3f0d15c879d9`). The smoke test (`experiments/results/smoke_test.json`) records 42 successful cases and the timings quoted in Section 5.

G Additional tables

Table 6: Acquisition policies (full predictor): Brier ALC [95% CI] and paired difference to the evaluator's random policy on the same pairs (negative favours the policy). The BLE rule accepts a candidate when its current prediction satisfies $|p - 0.5| \leq 0.15$ [Truong et al., 2026].

	Synthetic		Real	
Policy	ALC [95% CI]	Δ vs random [95% CI]	ALC [95% CI]	Δ vs random [95% CI]
random (evaluator default)	0.1797 [0.1691, 0.1900]	—	0.2224 [0.1949, 0.2454]	—
stream order	0.1784 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1915, 0.2400]	-0.0044 [-0.0107, 0.0019]
novelty filter (shipped)	0.1784 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1912, 0.2407]	-0.0044 [-0.0108, 0.0025]
novelty + uncertainty	0.1785 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2180 [0.1912, 0.2407]	-0.0044 [-0.0108, 0.0025]
uncertainty band (BLE rule)	0.1798 [0.1693, 0.1896]	+0.0001 [-0.0027, 0.0030]	0.2183 [0.1905, 0.2409]	-0.0041 [-0.0096, 0.0016]

Table 7: Real Ollama testbed: one pair per (model, harness, benchmark). Accuracy is the success rate over all items of the pair (acquisition and evaluation pools together).

Model (Ollama tag)	Harness	Benchmark	Items	Acc. (%)
qwen2.5:1.5b-instruct	8-shot CoT	GSM8K (test, first 200)	200	37.5
qwen2.5:3b-instruct	8-shot CoT	GSM8K (test, first 200)	200	24.0
qwen2.5:7b-instruct	8-shot CoT	GSM8K (test, first 200)	200	42.5
qwen2.5:1.5b-instruct	8-shot CoT	SVAMP (test, first 200)	200	79.0
qwen2.5:3b-instruct	8-shot CoT	SVAMP (test, first 200)	200	67.0
llama3.2:3b	direct answer	BBH mix (10 subtasks $\times$ 30)	300	49.3
qwen2.5:1.5b-instruct	direct answer	BBH mix (10 subtasks $\times$ 30)	300	39.3
qwen2.5:3b-instruct	direct answer	BBH mix (10 subtasks $\times$ 30)	300	29.3
qwen2.5:7b-instruct	direct answer	BBH mix (10 subtasks $\times$ 30)	300	51.0
llama3.2:3b	direct answer	GSM8K (test, first 200)	200	8.0
qwen2.5:1.5b-instruct	direct answer	GSM8K (test, first 200)	200	12.0
qwen2.5:3b-instruct	direct answer	GSM8K (test, first 200)	200	12.5
qwen2.5:7b-instruct	direct answer	GSM8K (test, first 200)	200	23.5
llama3.2:3b	direct answer	SVAMP (test, first 200)	200	33.0
qwen2.5:1.5b-instruct	direct answer	SVAMP (test, first 200)	200	32.5
qwen2.5:3b-instruct	direct answer	SVAMP (test, first 200)	200	39.5
qwen2.5:7b-instruct	direct answer	SVAMP (test, first 200)	200	64.5

Table 8: Brier score per budget for the main methods (random policy). The last column is the ALC of Eq. 1.

Testbed	Method	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
synthetic	constant 0.5	0.250	0.250	0.250	0.250	0.250	0.250	0.2500
	empirical mean	0.250	0.330	0.211	0.183	0.172	0.166	0.2208
	attribute prior only	0.215	0.215	0.215	0.215	0.215	0.215	0.2148
	prior + shrinkage	0.215	0.201	0.181	0.169	0.162	0.157	0.1797
	full (shipped)	0.215	0.201	0.181	0.169	0.162	0.157	0.1797
	oracle: pair success rate	0.160	0.160	0.160	0.160	0.160	0.160	0.1598
real	constant 0.5	0.250	0.250	0.250	0.250	0.250	0.250	0.2500
	empirical mean	0.250	0.498	0.264	0.231	0.217	0.202	0.2871
	attribute prior only	0.234	0.234	0.234	0.234	0.234	0.234	0.2340
	prior + shrinkage	0.234	0.247	0.226	0.213	0.211	0.201	0.2228
	full (shipped)	0.234	0.247	0.226	0.212	0.211	0.199	0.2224
	oracle: pair success rate	0.196	0.196	0.196	0.196	0.196	0.196	0.1961

Table 9: Synthetic signal sweep (160 pairs per setting, seed 0): ALC of shrinkage and of the full model, their paired difference, and the two oracle references.

σ_{topic}	γ	Shrinkage	Full [95% CI]	Full $-$ shrink [95% CI]	Oracle rate	Oracle p
0.00	0.0	0.1816	0.1818 [0.1708, 0.1926]	+0.0002 [-0.0000, 0.0005]	0.1543	0.1464
0.00	0.3	0.1801	0.1803 [0.1692, 0.1906]	+0.0002 [-0.0000, 0.0004]	0.1551	0.1444
0.35	0.3	0.1797	0.1800 [0.1689, 0.1905]	+0.0003 [0.0001, 0.0005]	0.1571	0.1427
0.70	0.3	0.1797	0.1797 [0.1691, 0.1900]	+0.0000 [-0.0002, 0.0003]	0.1598	0.1379
1.00	0.3	0.1800	0.1798 [0.1694, 0.1898]	-0.0002 [-0.0005, 0.0001]	0.1627	0.1321
0.70	0.0	0.1793	0.1796 [0.1688, 0.1896]	+0.0002 [0.0000, 0.0005]	0.1583	0.1392

Table 10: Sensitivity to the prior strength α_0 (Brier ALC).

	$\alpha_0 = 1$	$\alpha_0 = 2$	$\alpha_0 = 4$	$\alpha_0 = 8$
Synthetic, shrinkage	0.1829	0.1793	0.1797	0.1837
Synthetic, full	0.1831	0.1795	0.1797	0.1832
Real, shrinkage	0.2394	0.2290	0.2228	0.2208
Real, full	0.2388	0.2285	0.2224	0.2206